

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)

Факультет **Прикладная информатика**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №1

По дисциплине «Методы оптимизации»

Выполнил Тихонов Р.А. К3339

Санкт-Петербург 2026

Вариант 9

Задача: Максимизировать линейную ф-ию.

$$Z = 2x_1 + x_2 + 3x_3 + 2x_4 \rightarrow \max$$

При ограничениях:

$$\begin{cases} x_1 + 2x_2 + x_3 \leq 11 \\ x_1 + x_3 + x_4 = 8 \\ x_2 + x_4 \geq 3 \\ x_1, x_2, x_3, x_4 \geq 0 \end{cases}$$

1) Приведение к каноническому виду.

Умножим Z на -1 . Убавим от неравенств в ограничениях введенные доп. $x_5 \geq 0, x_6 \geq 0$

$$W(x) = -2x_1 - x_2 - 3x_3 - 2x_4 \rightarrow \min$$

$$\begin{cases} x_1 + 2x_2 + x_3 + x_5 = 11 \\ x_1 + x_3 + x_4 = 8 \\ x_2 + x_4 - x_6 = 3 \\ x_1, \dots, x_6 \geq 0 \end{cases}$$

2) Формулирование вспомогательной задачи

В каждом уравнении должна быть свободная переменная с коэф 1, которая встречается только в этом уравнении. В 1-м ур-ии это x_5 . В ~~2-м~~ 2-м и 3-м вводим искусственные $x_7 \geq 0$ и $x_8 \geq 0$.

Целевая ф-ция вспомогательной задачи

$$W(x) = x_7 + x_8 \rightarrow \min$$

$$\begin{cases} x_1 + x_2 + x_3 + x_5 = 11 \\ x_1 + x_3 + x_4 + x_7 = 8 \\ x_2 + x_4 - x_6 + x_8 = 3 \end{cases}$$

Выразим x_7 и x_8 :

$$x_7 = 8 - x_1 - x_3 - x_4$$

$$x_8 = 3 - x_2 - x_4 + x_6$$

$$\begin{aligned} \Rightarrow W'(x) &= (8 - x_1 - x_3 - x_4) + (3 - x_2 - x_4 + x_6) = \\ &= 11 - x_1 - x_2 - x_3 - 2x_4 + x_6 \end{aligned}$$

3) Решение вспомогательной задачи.

Составим начальную симплекс-таблицу:

Базис	x_1	x_2	x_3	x_4	x_6	b
x_5	1	2	1	0	0	11
x_7	1	0	1	1	0	8
x_8	0	1	0	1	-1	3
Δ	-1	-1	-1	-2	1	-11

3.1) Ведущий столбец - x_4 (т.к. -2 - min элемент)

3.2) $8:1=8$; $3:1=3 \Rightarrow$ Разрешающая строка - x_8

$\Rightarrow$ Разреш. элемент x_4 . Переменная x_4 входит в базис. Столбец x_8 исключаем.

Пересчитываем таблицу:

Базис	x_1	x_2	x_3	x_6	b
x_5	1	2	1	0	11
x_7	1	-1	1	1	5
x_4	0	1	0	-1	3
Δ	-1	1	-1	-1	-5

Разрешающий столбец x_1 (т.к. -1 - мин. элемент)
 $11/1 = 11$, $5/1 = 5 \Rightarrow x_2$ - Разрешающая строка
 Разреш. элемент -1. x_1 входит в базис,
 а x_2 исключаем.

Пересчитываем таблицу:

Базис	x_2	x_3	x_6	b
x_5	3	0	-1	6
x_1	-1	1	1	5
x_4	1	0	-1	3
Δ	0	0	0	0

Т.к. в строке нет отрицательных элементов,
 $W' = 0$, то исходная задача считается
 решенной.

4) Переход к задаче.

Выразим базисные переменные через свободные

$$x_1 = 5 + x_2 - x_3 - x_6$$

$$x_4 = 3 - x_2 + x_6$$

Подставляем в $W(x)$

$$W(x) = -2(5 + x_2 - x_3 - x_6) - x_2 - 3x_3 - 2(3 - x_2 + x_6)$$

$$W(x) = -16 - x_2 - x_3 \rightarrow \min$$

5) Решение основной задачи

Базис	x_2	x_3	x_6	b
x_5	3	0	-1	6
x_1	-1	1	1	5
x_4	1	0	-1	3
Δ	-1	-1	0	16

Разрешающий столбец x_2

Разрешающая строка x_5 ($6/3=2$; $3/1=3$)

Разрешающий элемент 3

Переменная x_2 входит в базис, x_5 ~~выходит~~ "выходит"

Базис	x_5	x_3	x_6	b
x_2	$\frac{1}{3}$	0	$-\frac{1}{3}$	2
x_1	$\frac{1}{3}$	1	$\frac{2}{3}$	7
x_4	$-\frac{1}{3}$	0	$-\frac{2}{3}$	1
Δ	$\frac{1}{3}$	-1	$-\frac{1}{3}$	18

Разреш. столбец: x_3

Разреш. строки: x_1

Разреш. элемент: 1

x_3 входит в базис, x_1 выхо

Пересчитывали таблицу

Базис	x_5	x_1	x_6	b
x_2	$\frac{1}{3}$	0	$-\frac{1}{3}$	2
x_3	$\frac{1}{3}$	1	$\frac{2}{3}$	7
x_4	$-\frac{1}{3}$	0	$-\frac{2}{3}$	1
Δ	$\frac{2}{3}$	1	$\frac{1}{3}$	25

В строке нет отриц. элементов $\Rightarrow$ задача решена.

Ответ:

Свободные переменные $x_1 = 0$

Базисные переменные равны значениям b .

$$x_2 = 2; \quad x_3 = 7; \quad x_4 = 1$$

Оптимальная точка $X^* = (0; 2; 7; 1)$

Значение целевой ф-ии: $Z_{\max} = 25$

6. Решение задачи в Excel

Задача была дополнительно решена в программе Excel с использованием надстройки «Поиск решения» симплекс-методом.

Процесс решения состоял из следующих этапов:

1. Ввод исходных данных

- **Блок переменных:** Для искомых значений переменных x_1, x_2, x_3, x_4 был выделен блок пустых ячеек A2:D2 (в них алгоритм должен поместить оптимальный ответ).
- **Целевая функция:** В ячейку E2 была введена формула для расчета значения целевой функции, ссылающаяся на ячейки переменных:
 $=2*A2 + B2 + 3*C2 + 2*D2$
- **Блок ограничений:** В ячейках A7, A8 и A9 были запрограммированы левые части системы ограничений. В них были введены следующие формулы:
 - Для 1-го ограничения: $=A2 + 2*B2 + C2$
 - Для 2-го ограничения: $=A2 + C2 + D2$
 - Для 3-го ограничения: $=B2 + D2$

2. Настройка параметров надстройки «Поиск решения»

После подготовки таблицы был запущен инструмент «Данные» → «Поиск решения», в котором были заданы следующие параметры математической модели:

- **Оптимизировать целевую функцию:** указана ячейка E2.
- **Направление оптимизации:** выбран переключатель «Максимум».
- **Изменяя ячейки переменных:** указан диапазон A2:D2.
- **Ограничения:** с помощью кнопки «Добавить» были заданы правые части и знаки для левых частей ограничений:
 - A7 <= 11
 - A8 = 8
 - A9 >= 3
- **Условие неотрицательности:** установлена галочка «Сделать переменные без ограничений неотрицательными» (выполнение условия $x_j \geq 0$).
- **Метод решения:** из выпадающего списка выбран алгоритм «Поиск решения линейных задач симплекс-методом».

Параметры поиска решения

Оптимизировать целевую функцию:

До: ☒ Максимум ☐ Минимум ☐ Значения:

Изменяя ячейки переменных:

В соответствии с ограничениями:

Добавить

Изменить

Удалить

Сбросить

Загрузить/сохранить

☒ Сделать переменные без ограничений неотрицательными

Выберите метод решения:

Параметры

Метод решения

Для гладких нелинейных задач используйте поиск решения нелинейных задач методом ОПГ, для линейных задач - поиск решения линейных задач симплекс-методом, а для негладких задач - эволюционный поиск решения.

Справка

Рис. 1: Настройка параметров

	A	B	C	D	E
1	x1	x2	xx3	x4	Z
2	0	2	7	1	25
3					
4					
5					
6					
7	11	<=	11		
8	8	=	8		
9	3	>=	3		

Рис. 2: Решение задачи с помощью надстройки "Поиск решения"

Программный расчет выдал точно такие же результаты: оптимальная точка $X^* = (0; 2; 7; 1)$ и максимальное значение функции $Z = 25$. Это полностью подтверждает абсолютную правильность моих ручных вычислений.

7. Программное решение

В качестве дополнительной проверки алгоритм симплекс-метода был написан с нуля на языке программирования Python. Созданный класс `Simplex` полностью автоматизирует ручной счет: приводит задачу к каноническому виду, решает вспомогательную и основную задачи, а также пересчитывает матрицы.

Исходный код программы:

```
import numpy as np
from fractions import Fraction as Frac

class Simplex:
    def __init__(self, func, limits, b_values, signs, find_max=True):
        self.func = [Frac(x) for x in func]
        self.limits = [[Frac(x) for x in row] for row in limits]
        self.b = [Frac(x) for x in b_values]
        self.signs = signs
        self.find_max = find_max

        self.rows_count = len(b_values)
        self.orig_vars_count = len(func)

        self.table = None
        self.basis = []
        self.isk_vars = []  # индексы искусственных переменных
        self.names = []

    def show_table(self, title):
        print(f"\n {title}")
        # Простой вывод заголовка
        head = "Базис | " + " | ".join(f"{n:>4}" for n in self.names) + " | b"
        print("-" * len(head))
        print(head)
        print("-" * len(head))

        for i in range(self.rows_count):
            current_basis = self.names[self.basis[i]]
            row_data = " | ".join(f"{str(val):>4}" for val in self.table[i, :-1])
            print(f" {current_basis:>4} | {row_data} | {str(self.table[i, -1]):>4}")

        print("-" * len(head))
        delta_data = " | ".join(f"{str(val):>4}" for val in self.table[-1, :-1])
        print(f" | {delta_data} | {str(self.table[-1, -1]):>4}")
        print("-" * len(head))

    def make_canonical(self):
        # Если ищем максимум, меняем знаки у функции
        self.c = [-x for x in self.func] if self.find_max else list(self.func)
        self.names = [f"x{i + 1}" for i in range(self.orig_vars_count)]

        dop_vars = []
        isk_vars = []

        # Разбираемся со знаками и добавляем новые переменные
        for i in range(self.rows_count):
            if self.b[i] < 0:
                self.b[i] *= -1
                self.limits[i] = [-x for x in self.limits[i]]
                if self.signs[i] == '<=':
                    self.signs[i] = '>='
                elif self.signs[i] == '>=':
                    self.signs[i] = '<='

            if self.signs[i] == '<=':
                dop_vars.append((i, 1))
                self.basis.append(len(self.names))
                self.names.append(f"x{len(self.names) + 1}")

            elif self.signs[i] == '>=':
                dop_vars.append((i, -1))
                self.names.append(f"x{len(self.names) + 1}")

            isk_vars.append(i)
            self.basis.append(len(self.names))
            self.isk_vars.append(len(self.names))
```

```

        self.names.append(f"x{len(self.names) + 1}")

    elif self.signs[i] == '=':
        isk_vars.append(i)
        self.basis.append(len(self.names))
        self.isk_vars.append(len(self.names))
        self.names.append(f"x{len(self.names) + 1}")

    # Создаем пустую таблицу (матрицу)
    total_vars = len(self.names)
    self.table = np.zeros((self.rows_count + 1, total_vars + 1), dtype=object)

    # Заполняем цифры
    for i in range(self.rows_count):
        self.table[i, :self.orig_vars_count] = self.limits[i]
        self.table[i, -1] = self.b[i]

    for idx, (row, val) in enumerate(dop_vars):
        self.table[row, self.orig_vars_count + idx] = Frac(val)

    for idx, row in enumerate(isk_vars):
        self.table[row, self.orig_vars_count + len(dop_vars) + idx] = Frac(1)

def recalculate(self, row, col):
    # Это правило прямоугольника
    element = self.table[row, col]
    self.table[row, :] = self.table[row, :] / element

    for i in range(len(self.table)):
        if i != row:
            self.table[i, :] -= self.table[i, col] * self.table[row, :]

    self.basis[row] = col # меняем базис

def run_iterations(self, phase):
    step = 1
    while True:
        self.show_table(f"{phase}. Шаг {step}")
        delta = self.table[-1, :-1]

        # Ищем самый большой минус
        min_val = min(delta)
        if min_val >= 0:
            print("Минусов больше нет. Конец этапа.")
            break

        col = np.argmin(delta)

        # Ищем строку по минимальному делению
        ratios = []
        for i in range(self.rows_count):
            if self.table[i, col] > 0:
                ratios.append(self.table[i, -1] / self.table[i, col])
            else:
                ratios.append(float('inf'))

        if min(ratios) == float('inf'):
            print("Ошибка: функция бесконечна!")
            return

        row = ratios.index(min(ratios))

        print(f"Разрешающий элемент: {self.table[row, col]}")
        print(f"Вместо {self.names[self.basis[row]]} заходит {self.names[col]}")

        self.recalculate(row, col)
        step += 1

def solve(self):
    self.make_canonical()

    # 1. Вспомогательная задача (если есть искусственные переменные)
    if len(self.isk_vars) > 0:
        print("\nРешаем Вспомогательную задачу:")
        for col in self.isk_vars:
            self.table[-1, col] = Frac(1)

    # Подставляем базис в строку дельта

```

```

        for i in range(self.rows_count):
            b_col = self.basis[i]
            if self.table[-1, b_col] != 0:
                self.table[-1, :] -= self.table[i, :] * self.table[-1, b_col]

        self.run_iterations("Фаза 1")

        if self.table[-1, -1] < 0:
            print("Решений нет!")
            return

        # Удаляем колонки искусственных переменных
        keep = [i for i in range(len(self.names)) if i not in self.isk_vars] + [-1]
        self.table = self.table[:, keep]
        self.names = [n for i, n in enumerate(self.names) if i not in self.isk_vars]

        # Сдвигаем индексы базиса
        for i in range(len(self.basis)):
            shift = sum(1 for art in self.isk_vars if art < self.basis[i])
            self.basis[i] -= shift

    # 2. Основная задача
    print("\nРешаем Основную задачу:")
    self.table[-1, :] = 0

    for i in range(self.orig_vars_count):
        self.table[-1, i] = self.c[i]

    for i in range(self.rows_count):
        b_col = self.basis[i]
        if self.table[-1, b_col] != 0:
            self.table[-1, :] -= self.table[i, :] * self.table[-1, b_col]

    self.run_iterations("Фаза 2")

    # 3. Вывод ответа
    print("\nФинальный ответ:")
    result = [Frac(0)] * self.orig_vars_count
    for i in range(self.rows_count):
        if self.basis[i] < self.orig_vars_count:
            result[self.basis[i]] = self.table[i, -1]

    print(f"Точка X* = ({', '.join(str(x) for x in result)})")

    final_z = self.table[-1, -1] if not self.find_max else -self.table[-1, -1]
    print(f"Максимальное значение Z = {final_z}")

# Подставлены параметры для 9 варианта
if __name__ == "__main__":
    func = [2, 1, 3, 2]

    limits = [
        [1, 2, 1, 0],
        [1, 0, 1, 1],
        [0, 1, 0, 1]
    ]
    b_values = [11, 8, 3]
    signs = ['<=', '=', '>=']

    task = Simplex(func, limits, b_values, signs, find_max=True)
    task.solve()

```

При запуске данный скрипт вывел в консоль все итерации симплекс-таблиц, полностью идентичные ручному счету. Финальный результат выполнения программы подтвердил ранее найденный ответ: оптимальная точка $X^* = (0, 2, 7, 1)$ со значением целевой функции $Z = 25$.

Рефлексивный вывод

Выполнение этой работы оказалось очень полезным: на лекциях симплекс-метод мне казался набором сложных формул, то ручной счет помог увидеть его понятную внутреннюю

логику. Самым трудным для меня было осознать метод искусственного базиса и не запутаться в дробях при пересчете таблиц. Однако, когда я разобрался с визуальным правилом «прямоугольника», алгоритм стал казаться простым и четким механизмом. Больше всего порадовало, что долгие ручные вычисления в итоге идеально совпали с секундным ответом от Excel.